UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, *Controller*, *Model*, dan *Middleware* pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Glean Vincent Rhamadhani
NPM	2428240132
Kelas	SI5C
Topik Tugas 1	Apotek
Entitas utama	Apotek
Tautan repositori	
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]` dan `[Entitas]`, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, `[sumber-daya]` menjadi `buku`.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan `POST`. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian *endpoint*	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau `curl` di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win + Shift + S`; macOS tekan `Cmd + Shift + 4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda `[...]` pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	
Entitas utama (tunggal)	
Alamat dasar	/
Field data	
Field wajib saat POST	
Query string untuk filter	
Data awal	
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey`.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

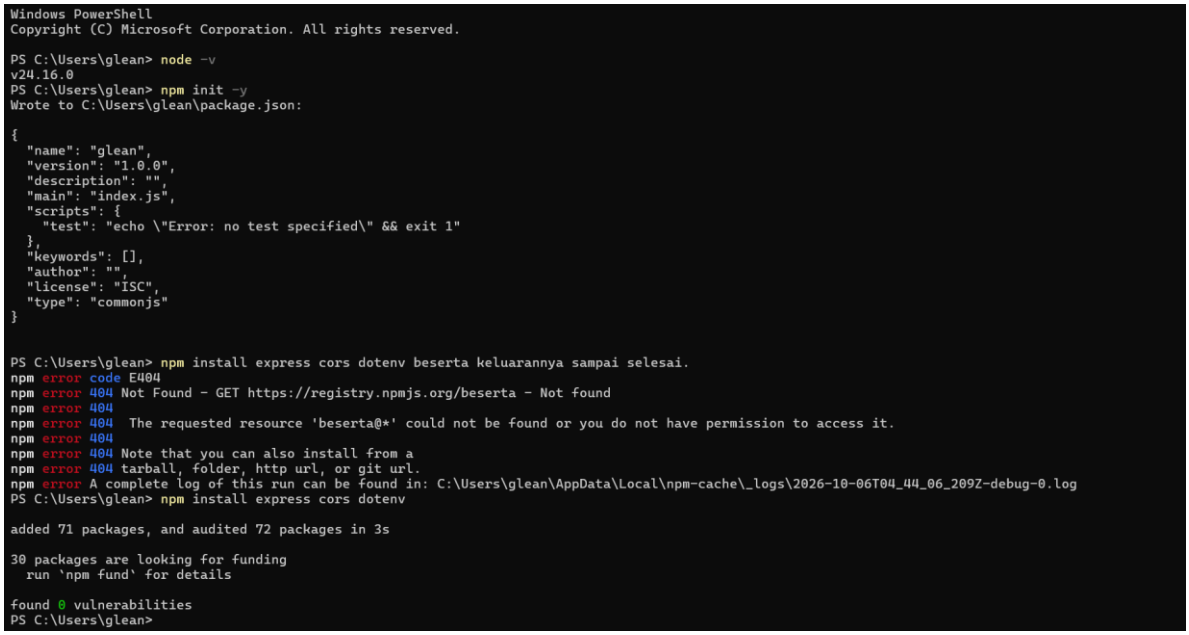
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

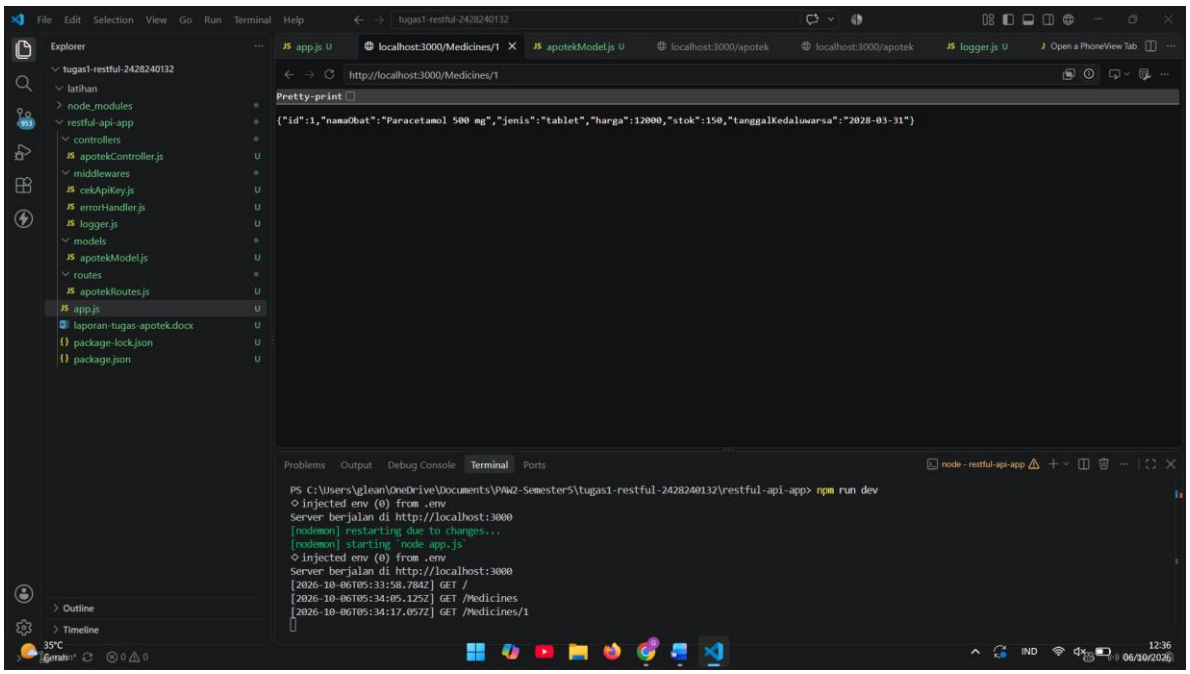
Jelaskan fungsi express, cors, dan dotenv, serta mengapa versi Node.js perlu memenuhi syarat Express 5.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder `routes`, `controllers`, `models`, dan `middlewares` beserta isinya. Gunakan panel *Explorer* editor atau perintah `tree`.



Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	
controllers/[sumber-daya]Controller.js	
routes/[sumber-daya]Routes.js	
middlewares/logger.js	
middlewares/cekApiKey.js	

Berkas	Penjelasan Anda
middlewares/errorHandler.js	
app.js	

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.

[Tempel screenshot di sini]

Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

> **Catatan**
> Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".
> Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

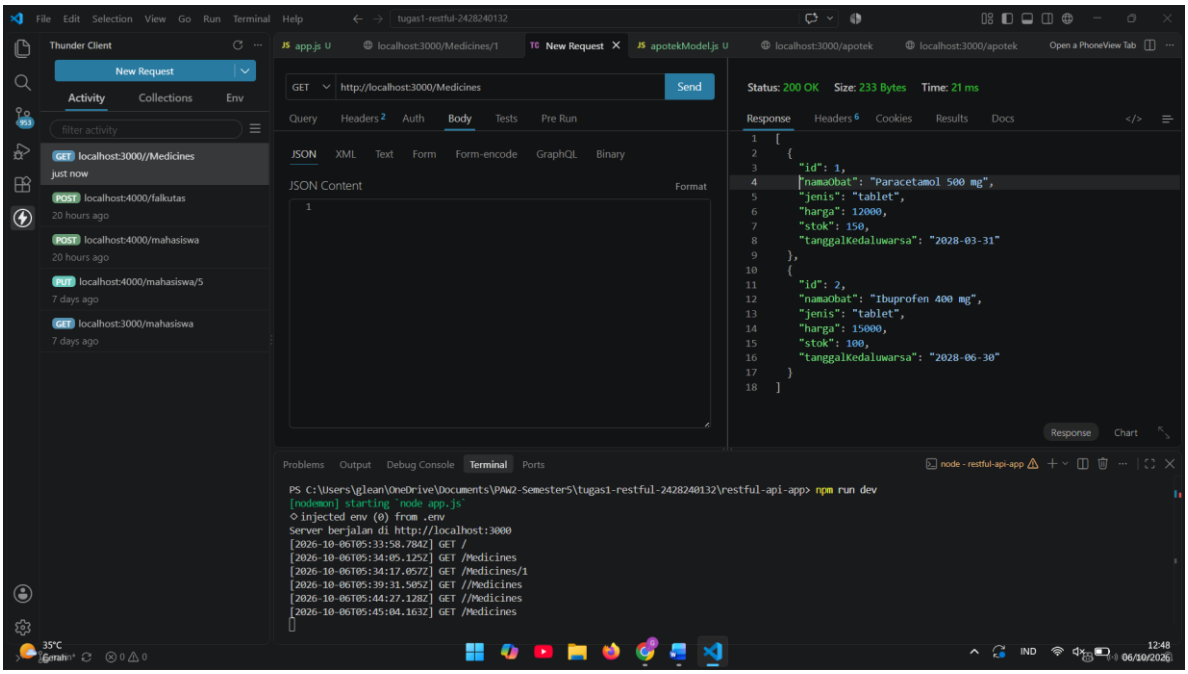
T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (*request*)	GET /http://localhost:3000//Medicines
Header	Thunder Client (https://www.thunderclient.com)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.

Status hasil uji	Status : 200 , [
	{
	"id": 1,
	"namaObat": "Paracetamol 500 mg",
	"jenis": "tablet",
	"harga": 12000,
	"stok": 150,
	"tanggalKedaluwarsa": "2028-03-31"
	},
	{
	"id": 2,
	"namaObat": "Ibuprofen 400 mg",
	"jenis": "tablet",
	"harga": 15000,
	"stok": 100,
	"tanggalKedaluwarsa": "2028-06-30"
	}
	]

Lulus?	[ ] Ya [ ] Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

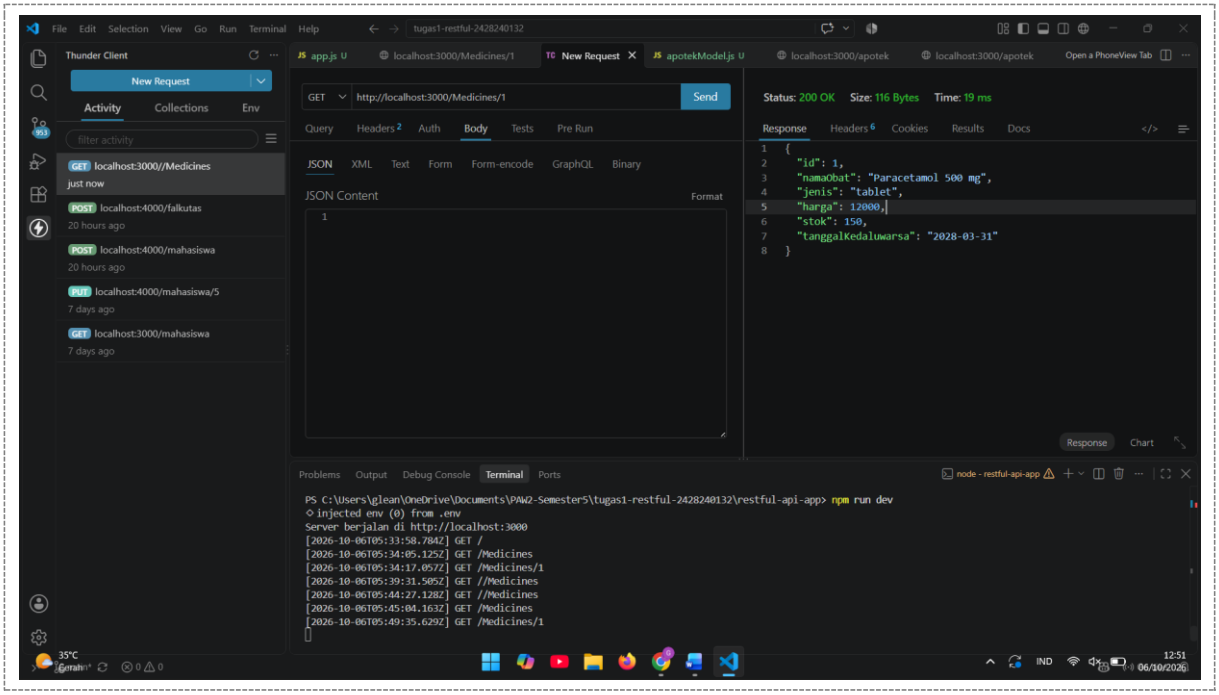
T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (*request*)	GET http://localhost:3000/Medicines/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.

Status hasil uji	{
	"id": 1,
	"namaObat": "Paracetamol 500 mg",
	"jenis": "tablet",
	"harga": 12000,
	"stok": 150,
	"tanggalKedaluwarsa": "2028-03-31"
	}

Lulus?	[ ] Ya [ ] Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

Jelaskan bagaimana ID dari alamat diambil (req.params) dan mengapa perlu diubah menjadi angka.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (*request*)	POST /[sumber-daya]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field wajib 1]": "...", "[field wajib 2]": "..." }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]
Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

Penjelasan:
Jelaskan peran header x-api-key (middleware apa yang memeriksanya), peran express.json(), dan alasan status yang dipakai adalah 201, bukan 200.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (*request*)	PUT /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

Penjelasan:

Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (*request*)	DELETE /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (*request*)	POST /[sumber-daya] (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

Penjelasan:

Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (*request*)	POST /[sumber-daya]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (*request*)	POST /[sumber-daya]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

Penjelasan:

Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (*request*)	GET /[sumber-daya]/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya [ ] Tidak

[Tempel screenshot di sini]

Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	
Pesan commit terakhir	
Folder routes, controllers, models, middlewares ada	[ ] Ya [ ] Tidak
.env tidak terunggah (ada di .gitignore)	[ ] Ya [ ] Tidak
.env.example ada	[ ] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:
Tulis jawaban Anda di sini.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:
Tulis jawaban Anda di sini.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan `.env` tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; `.env` tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		**100**